

**REFLEKSI TUGAS OBJECT-ORIENTED PROGRAMMING
SISTEM LAUNDRY**



I GUSTI PUTU ARI ARCHELLYNO (2505551116)

IDA BAGUS NYOMAN ANANTA WIBAWA PUTRA (2505551005)

PEMROGRAMAN BERORIENTASI OBYEK (D)

PROGRAM STUDI SARJANA TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS UDAYANA

2025

PERTANYAAN

1. AI tool apa yang digunakan dan mengapa memilih *tool* tersebut?
2. Apa saja saran/perubahan yang diberikan oleh AI terhadap kode fase 1?
3. Saran mana yang kalian terima dan mengapa?
4. Saran mana yang kalian tolak dan mengapa?
5. Apakah ada saran dari AI yang salah atau kurang tepat? Jika ya, jelaskan.
6. Konsep OOP apa yang kalian pahami lebih baik setelah proses review dengan AI?
7. Bagaimana pembagian tugas antar anggota kelompok pada setiap fase?
Dibuat dalam bentuk tabel.

JAWABAN

1. Kelompok kami menggunakan Claude (claude.ai) yang dikembangkan oleh Anthropic sebagai AI *tool* utama dalam proses review kode. Kami memilih Claude dengan beberapa pertimbangan utama.
 - a. Pertama, Claude memiliki kemampuan yang baik dalam memahami kode Java dan memberikan penjelasan yang terstruktur. Ketika kami mengajukan pertanyaan tentang konsep OOP, Claude tidak hanya memberikan jawaban benar atau salah, tetapi juga menjelaskan alasan di balik setiap saran secara mendalam. Ini sangat membantu untuk proses belajar, bukan sekadar memperbaiki kode.
 - b. Kedua, Claude tersedia secara gratis melalui antarmuka web claude.ai, sehingga teman kelompok dapat mengaksesnya tanpa biaya tambahan.
 - c. Ketiga, Claude mampu menjawab pertanyaan dalam bahasa Indonesia dengan baik, sehingga memudahkan diskusi.
2. Setelah kami mengajukan pertanyaan-pertanyaan spesifik kepada Claude, berikut saran utama yang diberikan terhadap kode Fase 1:
 - a. Masalah *Encapsulation*: Atribut pada *class* Pelanggan dan Pesanan tidak memiliki *access modifier private*, sehingga bersifat *package-*

private dan bisa diakses langsung dari luar *class*. AI menyarankan penambahan keyword *private* pada semua atribut.

- b. Bug Perbandingan String: Penggunaan operator `==` untuk membandingkan String di *method* `hitungBiaya()` dan `cariPesanan()` merupakan bug kritis yang menyebabkan pencarian pesanan tidak pernah berhasil. AI menyarankan penggunaan *method* `.equals()`.
 - c. Desain Enum untuk Jenis Layanan: Menggunakan String bebas untuk jenis layanan rawan typo. AI menyarankan pembuatan enum `JenisLayanan` yang juga menyimpan harga per kg, sehingga perhitungan biaya lebih terstruktur.
 - d. Akses *Modifier Method*: *Method* `hitungBiaya()` seharusnya *private* karena merupakan logika internal, tidak perlu diakses dari luar *class*.
 - e. Penanganan Input: Program dapat *crash* jika pengguna mengetik huruf saat diminta input angka. AI menyarankan penggunaan *try-catch* untuk `InputMismatchException`.
 - f. Format Output: Angka biaya ditampilkan sebagai 25000.0 yang kurang rapi. AI menyarankan penggunaan `printf` dengan format `%,.0f` agar tampil sebagai 25.000.
 - g. Penggunaan Interface List: AI menyarankan deklarasi `daftarPesanan` menggunakan tipe List (*interface*) daripada `ArrayList` (konkret) untuk fleksibilitas.
3. Sebagian besar saran dari AI kami terima karena secara langsung memperbaiki bug nyata atau meningkatkan kualitas kode sesuai standar Java.
- a. Penambahan *private* pada atribut kami terima karena ini adalah inti dari konsep encapsulation yang memang sedang kami pelajari. Tanpa *private*, *getter* dan *setter* yang kami buat menjadi tidak bermakna karena atribut tetap bisa diakses langsung.

- b. Perbaikan bug `==` menjadi `.equals()` kami terima karena ini adalah bug nyata yang kami temukan sendiri saat pengujian karena pencarian pesanan tidak berfungsi. Penjelasan AI tentang perbedaan referensi objek dan nilai string sangat memperjelas mengapa bug ini terjadi.
 - c. Penggunaan enum `JenisLayanan` kami terima karena menyederhanakan kode dan menghilangkan risiko typo. Dengan enum, konstanta harga juga tersimpan bersama jenisnya, sehingga kode lebih terorganisir.
 - d. Penambahan *try-catch* kami terima karena membuat program lebih *robust*. Kami pernah mengalami program *crash* saat pengujian, dan solusi AI langsung menyelesaikan masalah tersebut.
4. Terdapat dua saran dari AI yang kami putuskan untuk tidak kami terapkan pada Fase 2.
- a. Pertama, saran untuk mendeklarasikan `daftarPesanan` menggunakan tipe *interface* `List (List<Pesanan>)` daripada `ArrayList<Pesanan>` secara langsung. Meskipun AI menjelaskan prinsip '*program to an interface*' dengan baik, kami menilai konsep ini lebih cocok untuk proyek skala besar. Untuk program *console* sederhana dengan satu jenis koleksi, penggunaan `ArrayList` secara langsung sudah cukup dan lebih mudah dipahami oleh anggota kelompok yang masih belajar. Menerapkannya justru dapat membuat kode terlihat lebih kompleks dari yang diperlukan.
 - b. Kedua, saran untuk mengubah nama method `updateStatus()` menjadi `perbaruiStatus()` agar konsisten dengan penamaan bahasa Indonesia. Kami memutuskan mempertahankan `updateStatus()` karena kata '*update*' sudah sangat umum digunakan dalam konteks pemrograman, mudah dipahami, dan tidak mengurangi keterbacaan kode. Konsistensi bahasa dalam penamaan memang penting, tetapi kami menilai hal ini bukan prioritas utama mengingat *scope* tugas.

5. Secara keseluruhan, saran-saran yang diberikan Claude akurat dan relevan. Tidak ada saran yang kami temukan sebagai salah secara teknis. Namun, ada satu catatan:
 - a. Ketika AI menyarankan penggunaan enum `JenisLayanan`, AI tidak secara eksplisit menyebutkan bahwa perubahan ini mengharuskan pembaruan pada class `Main.java` (cara membaca *input* dari pengguna harus berubah, tidak lagi menerima String bebas). Kami baru menyadari dampak ini saat mengimplementasikan saran tersebut. Ini bukan saran yang salah, tetapi kurang lengkap dari segi dampak perubahan ke bagian lain kode. Pelajaran yang kami petik: selalu pertimbangkan dampak perubahan desain ke seluruh bagian program, tidak hanya *class* yang sedang dimodifikasi.
6. Proses review dengan AI memberikan pemahaman yang lebih mendalam terhadap beberapa konsep OOP penting.
 - a. Konsep *encapsulation* menjadi jauh lebih jelas setelah AI menjelaskan bahwa hanya menambahkan *getter/setter* tanpa menjadikan atribut *private* tidak berarti *class* sudah ter-*encapsulate* dengan baik. Kami memahami bahwa *private* adalah kunci *encapsulation* yang sesungguhnya. Tanpanya, seluruh mekanisme *getter/setter* tidak bermakna karena data tetap bisa diakses langsung.
 - b. Kami juga memahami lebih dalam tentang prinsip *single responsibility* setiap *method* sebaiknya memiliki satu tanggung jawab yang jelas. *Method* `hitungBiaya()` yang semula *public* padahal hanya digunakan secara internal menjadi contoh nyata pentingnya memikirkan *visibility* setiap *method*.
 - c. Penggunaan enum sebagai tipe data yang membawa perilaku (bukan sekadar konstanta) juga merupakan wawasan baru yang memperkaya pemahaman kami tentang OOP di Java. Dimana enum pun bisa memiliki *constructor*, *method*, dan atribut layaknya *class* biasa.

7. Pembagian tugas antar anggota kelompok:

Anggota	Fase 1	Fase 2	Fase 3 (Log AI & Refleksi)
Ananta	Membuat menu interaktif, KasirLaundry.java dan logika bisnis	Memperbaiki Main.java, <i>encapsulation</i> & bug == membuat enum JenisLayanan	Menulis laporan refleksi bagian 1-3
Archell	Merancang struktur <i>class</i> keseluruhan, membuat Main.java, Pelanggan.java & Pesanan.java	Memperbaiki KasirLaundry.java, menambahkan <i>try-catch</i> , menambahkan <i>method</i> baru	Menulis laporan refleksi bagian 4-7